

Assumptions of an MDPs: Markov Properties (3)

Why discount rewards?

(1) recurrent states (2) Transient states (3) Absorbing states

Problems of greedy policy improvement

Pros & cons of SARSA & Q-learning

Bag of tricks to stabilise learning DQNs

1) Experience replay

Prob: exp. correlated over time

Sol: random sampling of experience mini batches

- sample efficiency
- reduction of var. due to broken correlations.

2) Separate, frozen Q target

Prob: Target Q values

$$y_i = r + \gamma \max_{a' \in A} \hat{Q}(s', a'; w_{i-1}) \quad \text{change constantly.}$$

Oscillations & divergence during training.

Sol: 2 Q-networks

- 1) To predict Q-targets with param w^- (Frozen Target Q-n/w)
- 2) To evaluate current Q values with param w (Dynamic main Q-n/w)

- reduces chance of divergence
- avoids oscillations by having fixed targets.

3) Reward clipping

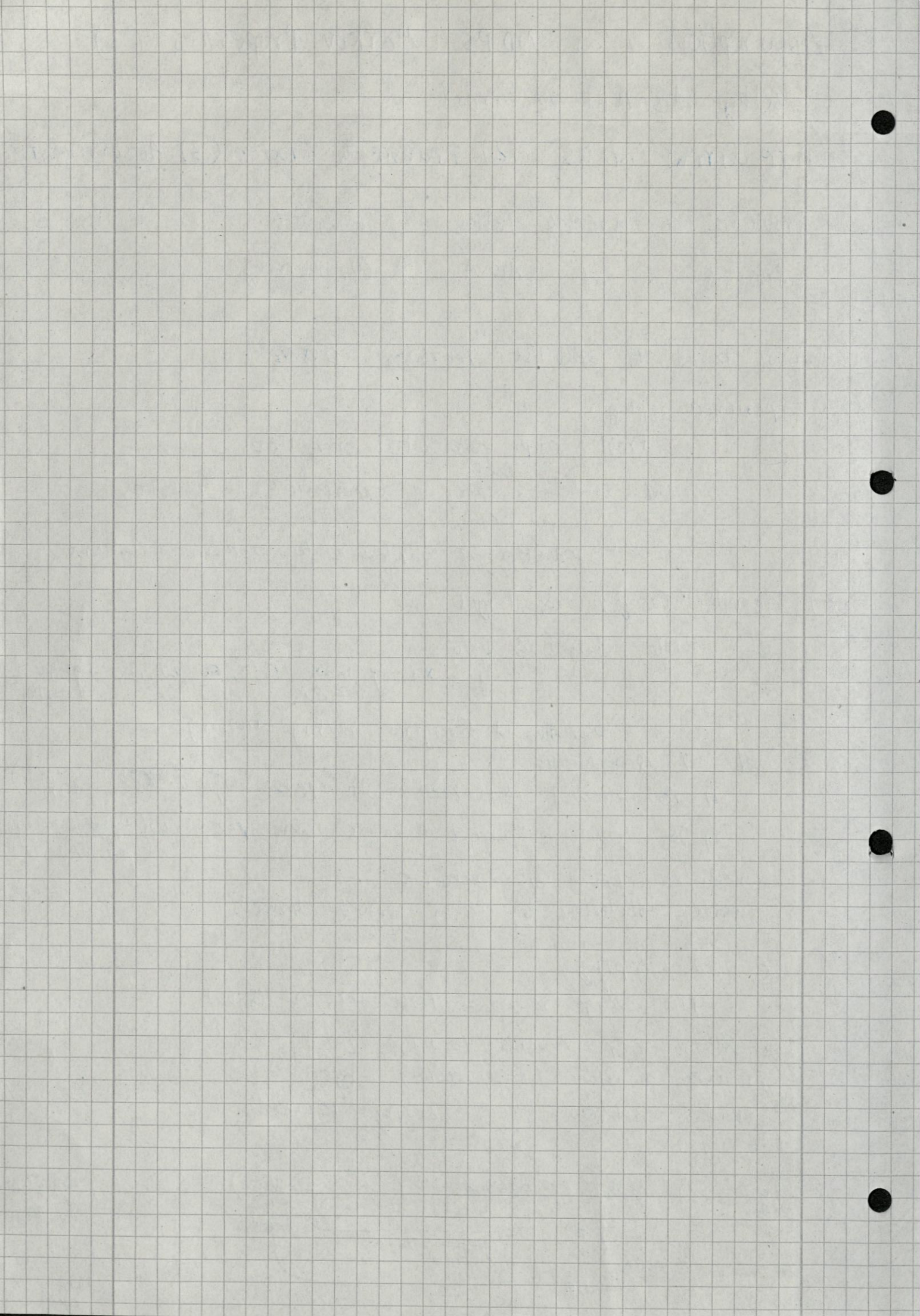
Prob: large rewards = large var. in Q-val

Sol: clip rewards to range $[-1, 1]$

- prevents Q values from becoming large
- avoid oscillations by normalizing rewards.

$$y_i = r + \gamma \hat{Q}(s', \underset{\substack{\text{action} \\ \text{evaluation}}}{\arg \max_{a' \in A}} \hat{Q}(s', a'; \theta_i); v_i) ; \quad \delta = y_i + \gamma \max_{a'} \hat{Q}(s, a) - \hat{Q}(s, a)$$

$|\delta| \rightarrow$ absolute bellman error



Policy based RL

- VFA parametrised the value func. & then derived the policy ex: E-greedy

- Policy based RL parametrises the policy directly.

$$\pi_\theta(a|s) = p(a|s; \theta)$$

Goal: Find θ that maximises long term reward.

+ good convergence

+ easily extend to high-dim / cont. action / state spaces.

+ can learn stochastic policies

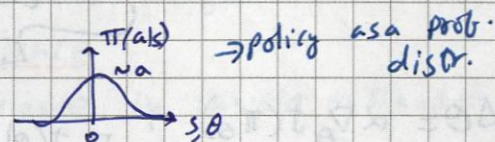
+ policies are simple

- susceptible to local optima

- obtained knowledge is specific & doesn't generalize well

- ignores a lot of ^{info in} data.

stochastic policy: $\pi(a|s) = p[a|s; \theta]$



Need for stochastic policies: (1) Deterministic policies are easily exploited (rock paper scissors)

(2) Agent cannot differentiate similar states (Aliased grid world)

Why learn policy directly? → Greedy updates cause large policy jumps, potentially leading to unstable learning process. $\theta_{n+1} = \arg \max_{\theta} E_{\pi_{\theta}} \{ Q^{\pi}(s, a) \}$
→ Policy grad. methods have smooth policy improvement for stable learning process

$$\theta_{n+1} = \theta_n + \alpha \nabla_{\theta} J(\theta_n) ; J = \sum_{t=0}^{\infty} \gamma^t r_t$$

FDSA (Finite Difference Stochastic approach) to evaluate gra.

$$\frac{\partial J(\theta)}{\partial \theta_k} \approx \frac{J(\theta + \epsilon u_k) - J(\theta)}{\epsilon} ; u_k \rightarrow \text{unit vector in } k^{\text{th}} \text{ comp.}$$

Prob: (1) Req. lot of samples (2) in stochastic env., small ϵ_n it is hard to distinguish diff. w/ R^+ & R^-

→ Augmented Random Search (ARS); Builds on Basic Random Search to find parameter improvements in θ .

idea: perturb θ using small +ve/-ve changes $\pm \delta$, run simulations to collect rewards using the 2 new param. then update param. based on diff. in collected rewards.

3 improvements by ARS: 1) divide rewards by std. deviation σ_r
2) Normalise states
3) use only top-k best rollouts to compute avg.

$$\theta_{j+1} = \theta_j + \frac{\alpha}{N} \sum_{k=1}^N [r^+ - r^-] \delta_k$$

* Robust to hyper-parameters & min tuning req.

* Easy to parallelise

* Simple to understand & implement

- favour lucky rollouts

- don't exploit seq. structure

- require 10x samples

- not easy in stochastic env. to distinguish b/w good perf due to param. variation or noise.

Policy gradients:

- State-action seq. in complete trajectory with T steps

$$\tau = (s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T) \rightarrow \text{trajectory}$$

$R(s_t, a_t) \rightarrow$ reward after observing s_t & performing a_t

goal: maximise expected rewards

$$\max_{\theta} E_{\pi_{\theta}} G(\tau)$$

- Use gradient ascent. calc. gradient WRT parameters

$$\nabla_{\theta} E_{\tau \sim \pi_{\theta}} G(\tau) ; \theta \leftarrow \theta + \alpha \nabla_{\theta} E_{\tau \sim \pi_{\theta}} G(\tau)$$

$\rightarrow$ policy gradient also = $\nabla_{\theta} J(\pi_{\theta})$

$$\Delta \theta = \alpha \nabla_{\theta} J(\pi_{\theta}) ; \nabla_{\theta} J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$

Through some derivation

$$\begin{aligned} \nabla_{\theta} E_{\tau \sim \pi_{\theta}} G(\tau) &= E_{\tau \sim \pi_{\theta}} \left(\nabla_{\theta} \log p(\tau | \theta) G(\tau) \right) \\ &= E_{\tau \sim \pi_{\theta}} \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(\tau) \right) \end{aligned}$$

$\rightarrow$ intuition here is - increase prob. of paths with +ve G
- decrease " " " " -ve "

$\rightarrow$ this expectation is estimated by sample mean using MC sampling of $N = L$

$$\begin{aligned} \nabla_{\theta} E_{\tau \sim \pi_{\theta}} G(\tau) &\approx \frac{1}{L} \sum_{\tau} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(\tau) \\ &\approx \frac{1}{L} \sum_{\tau} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T \gamma^{t'-t} R(s_{t'}, a_{t'}) \end{aligned}$$

$\rightarrow$ each action in episode influenced by reward of whole episode?

$\rightarrow$ reward to go

* each action in episode is not influenced by reward of whole episode, **policy grad.**
instead action taken at time t uses "return-to-go" G_t which is sum of disc. rewards from $t+1$ to end of episode.

* this method helps reduce variance of policy grad. estimate compared to using full episode reward for every action.

REINFORCE: MC policy gradient control (episodic)

i/p: differentiable policy parametrisation $\pi(a|s, \theta)$

param: step size $\alpha > 0$

init policy param $\theta \in \mathbb{R}^{d'}$ (ex: to 0)

loop forever (loop for each episode):

Generate an episode $s_0, a_0, R_1, \dots, s_{T-1}, a_{T-1}, R_T$ from $\pi(\cdot | \cdot, \theta)$

loop for each step of episode $t=0, 1, \dots, T-1$

$$G \leftarrow \sum_{k=t}^T \gamma^{k-t} R_k$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(a_t | s_t, \theta)$$

Variance & Baselines

Expected Grad-log-Prob lemma:

$p_\theta \rightarrow$ parametrised prob. distr. over random var. x .

$$E_{x \sim p_\theta} [\nabla_\theta \log p_\theta(x)] = 0$$

$\Rightarrow$ For any func. v that depends on states

$$E_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) v(s_t)] = 0$$

Hence we can +/- any such terms (baseline functions) to policy grad. w/o changing its expectation

$$\nabla_\theta E_{x \sim \pi_\theta} G(x) = E_{x \sim \pi_\theta} \left(\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) (G_t - \underline{v(s_t)}) \right)$$

How does this help? $\rightarrow$ introduces baselines to our target

$\rightarrow$ define advantage func. to reduce variance.

Ex: use $v(s_t) = V^\pi(s_t)$ to reduce variance in sample estimate of policy gradient.

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) \rightarrow \text{faster \& more stable policy learning.}$$

REINFORCE with baselines

I/P: a differentiable policy parametrisation $\pi(a|s, \theta)$

a " state-value func. parametrisation $\hat{v}(s, w)$

param: step sizes $\alpha^\theta > 0$ & $\alpha^w > 0$

Init: policy param. $\theta \in \mathbb{R}^{d'}$ & state value weights $w \in \mathbb{R}^d$

loop: for each episode/forever

Gen. an episode $s_0, a_0, R_1, \dots, s_{T-1}, a_{T-1}, R_T$ following $\pi(\cdot | \cdot, \theta)$

for each step of @episode $t = 0, 1, \dots, T-1$

$$G_t \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

$\rightarrow$ advantage.

$$\delta \leftarrow G_t - \hat{v}(s_t, w)$$

$$w \leftarrow w + \alpha^w \delta \nabla \hat{v}(s_t, w)$$

$$\theta \leftarrow \theta + \alpha^\theta \gamma^t \delta \nabla \ln \pi(a_t | s_t, \theta) \rightarrow \text{policy grad update}$$

Actor-Critics

- Recall MC policy grad. is sampled & has high var.

$$\nabla_\theta E_{\pi_\theta} G(x) \approx \frac{1}{L} \sum_{i=1}^L \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \underbrace{\sum_{t'=t}^T \gamma^{t'-t} R(s_{t'}, a_{t'}) - v(s_t)}_{Q^\pi(s_t, a_t)}$$

idea: use a critic to estimate Q

- in practice we can't compute $Q^\pi(s_t, a_t)$ hence approximate it with a NN with param. ϕ & std. SGD over k epochs minimising the MSE:

$$\phi_k = \arg \min_{\phi} E_{s_t, a_t \sim \tilde{\pi}_k} [Q_\phi(s_t, a_t) - \hat{R}_t]^2$$

Hence, Actor: updates θ for policy gradient, i.e. it updates the policy.

Critic: evaluates actions taken by the actor. It estimates value function (V/Q) or Advantage function (A-value). It updates its parameters ϕ by minimising MSE of its value estimates, through methods like n-step TD.

Typically we estimate $v^\pi(s_t; \phi)$ explicitly A then sample $q^\pi(s_t, a_t) \approx G_t^{(n)}$

Ex: $\hat{G}_t^{(1)} = R_t + \gamma v^\pi(s_{t+1}; \phi)$

$$\Rightarrow \nabla_{\theta} E_{\pi_{\theta}} \hat{G}_t = \frac{1}{L} \sum_L \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (\hat{G}_t - v(s_t))$$

↳ i.e. we use approx grad. in direction suggested by critic.

→ calc. advantage function $A^\pi(s_t, a_t) = \hat{G}_t - v(s_t)$ is also easy

$$\begin{aligned} A^\pi(s_t, a_t) &= Q^\pi(s_t, a_t) - v^\pi(s_t) \\ &= r + \gamma v^\pi(s_{t+1}) - v^\pi(s_t) \end{aligned}$$

- ↳ for G_t
- full returns = high var
 - 1 step TD = high bias
 - n " " = in between

→ to get rid of var. use on-policy targets. (i.e. can be corrected via importance sampling)

- deals with cont. action spaces, Gaussian policy is common

$$a \sim \mathcal{N}(\mu(s), \sigma^2)$$

Can be used in REINFORCE / actor-critic

Exploration v/s Exploitation for PG Methods

- PG trains stochastic policy in an on-policy way
- actions sampled from ^{env. from} latest stochastic policy.
- randomness in selecting actions; note initially A drops with further training.
- easy to get stuck in local min. PG considers imp under current data.
- can use ϵ -greedy but not optimal.
- alternatively ensure entropy of policy isn't too low

$$-\sum_s \mu(s) \sum_a \pi(a|s) \log \pi(a|s) = -E[\log \pi(a_t | s_t)]$$

add a regularisation term that pushes entropy up slightly each step. Similar to KL reg. in TRPO.

- May increase var. in Gaussian policies
- makes softmax more uniform.

- off policy learning not possible

Policy grad has many forms

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) G_t]$$

↳ REINFORCE

- critic uses policy eval. (MC/TD) to estimate

$$Q^\pi(s, a) / v^\pi(s)$$

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) (G_t - v(s_t))]$$

↳ reinforce w baseline

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \hat{A}_t^{(\infty)}] \rightarrow \text{advantage actor-critic}$$

recall goal: $\max_{\theta} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}} \left(\sum_{t=0}^{\infty} \gamma^t r_t \right)$

policy grad: $\nabla_{\theta} J(\pi_{\theta}) = \nabla_{\theta} E_{\pi_{\theta}} h(\tau) \approx \frac{1}{2} \sum_{\tau} \sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t = g$

$\hat{A}_t > 0$ (< 0): grad becomes +ve (-ve)

prob. of taking a_t in s_t is increased (decreased)

$\theta_{k+1} = \theta_k + \alpha g$
↳ policy gradient

General template for PG algos

Loop forever

collect traj via policy π_{θ}

calc. adv. function at each timestep $A^{\pi_{\theta}}(a_t | s_t)$

compute estimated policy grad.

$$\nabla J(\theta) \sim E_{\tau \sim \pi_{\theta}} \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi_{\theta}}(a_t | s_t) \right)$$

Update policy param $\theta_{\text{new}} \leftarrow \theta + \alpha \nabla J(\theta)$

end for

→ Adv. function is a noisy estimate A might not be accurate.

→ def. A scheduling of LR in RL is tricky.

→ 1st order derivatives approximate param. surface to be flat
if it has high curvature it is dangerous.

→ bad policy leads to bad data unlike in value based RL!

TRPO

given: $\theta \leftarrow \theta + \alpha \nabla J(\theta)$ $A \pi_{\theta}(a | s)$

- some param. change more than others.

- single value for α isn't sufficient

what we then do is

$$\theta' \leftarrow \underset{\theta'}{\operatorname{argmax}} (\theta' - \theta)^T \nabla J(\theta) \text{ subject to } \|\theta' - \theta\|^2 \leq \epsilon$$

$$\underbrace{D(\pi_{\theta'}, \pi_{\theta}) \leq \epsilon}_{\text{Trust region}}$$

idea:

- regularize updates to policy param, such that
policy doesn't change too much.

Problem of PG

- keeps old & new policy close in param. space

- small changes lead to large diff. in performance

- large step sizes hurt performance.

→ we want improvement but not degradation

if we want to optimise $L(\pi_{\text{new}})$ instead of $J(\pi_{\text{new}})$ with a guarantee of monotonic improvement on $J(\pi_{\text{new}})$ we need a bound on $L(\pi_{\text{new}})$. The foll. bound holds

$$J(\pi_{\text{new}}) \geq L(\pi_{\text{new}}) - C \cdot D_{\text{KL}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}); \quad C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$$

$$D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\text{new}}) = \mathbb{E}_{\pi_{\text{old}}(a)} \log \frac{\pi_{\text{old}}(a)}{\pi_{\text{new}}(a)}$$

- We approximate expected return locally around current policy. Accuracy decreases if new & current policies diverge.
- can establish upper bound for error & can guarantee J impr. if we optimise the local approximation within a trusted region.

Draw backs:

- TRPO minimises a quadratic eq. to approximate inverse Fisher Info Matrix & to do this for every policy is expensive.
- requires large batch of rollouts to approximate it correctly.
- less sample efficient than other PG methods.
- ↳ these become significant for deep networks.

PPO

Motivation same as for TRPO

- Make biggest improvement step & not too far such that perf. collapses

Addresses shortcomings of TRPO

- PPO uses 1st order methods with a few tricks v/s TRPO using 2nd order
- simpler to implement & has similar perf. to TRPO empirically.

2 variants

- 1) PPO-penalty: TRPO with KL-penalization instead of constraint (penalty adjusted & scaled over training - adaptive KL penalty coefficient)
- 2) PPO-clip: No constraints! Add clipping to objective func. to remove incentives to move too far.

- TRPO minimises a surrogate objective s.t. to a constraint on size of policy update

$$\max_{\theta} \hat{E}_{\pi_{\theta}} \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} \hat{A}_t \right] \quad \text{s.t.} \quad \hat{E}_{\pi_{\text{old}}} [\text{KL}[\pi_{\theta}(\cdot | s_t), \pi_{\text{old}}(\cdot | s_t)]] \leq \delta$$

↳ definition of trust region

↳ according to advantage we want to increase prob. of taking certain actions in a state in new policy divided by prob. of taking the action in old policy.

⊛ we want the 2 policies to be close to each other in policy space.

intuition (TRPO): - measure how π_{θ} performs relative to π_{old} using data from old policy.

To reformulate for PPO

- Define prob. ratio $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, i.e. $r_t(\theta_{old}) = 1$

- In other words TRPO maximises the foll. objective

$$L^{CPI}(\theta) = \hat{E}_t [r_t(\theta) \hat{A}_t] \quad \text{CPI} \rightarrow \text{conservative policy iteration}$$

As we're penalizing changes to policy that move $r_t(\theta)$ far away from 1

- Finally the PPO objective we want to maximise is

$$L(\theta) = \hat{E}_t [\min(\underbrace{r_t(\theta) \hat{A}_t}_{L^{CPI}(\theta)}, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

where ϵ is hyperparam that decides how far π_{new} may go away from π_{old} .

2nd term inside min clips prob. ratio & removes the incentive for moving r_t outside of the interval $[1-\epsilon, 1+\epsilon]$

→ we take the min of clipped & unclipped objective

→ the final objective is a lower bound on the unclipped obj.

→ the clipping operator is a pessimistic bound of unclipped objective

negative prob. ratio $r_i(a) = \frac{\pi(a|x_i)}{\pi(a)}$

In order to find the best hypothesis

$$f(a) = E[f(a|x)] = \sum_i p(x_i) f(a|x_i)$$

where $f(a|x_i)$ is the probability of a given x_i

Finally, the best hypothesis is the one that

$$f(a) = E[f(a|x)] = \sum_i p(x_i) f(a|x_i)$$

where $f(a|x_i)$ is the probability of a given x_i

In order to find the best hypothesis, we need to find the value of a that maximizes $f(a)$

we take the derivative of $f(a)$ with respect to a

is the final objective is a linear function of the variables x_i

The clipping operator is a basic arithmetic operation of unclipped objective

Exploration & Exploitation

- we were improving π with $\pi' = \arg\max_{a \in A} Q^\pi(s, a)$ which had problem of bootstrapping Q -function
- ϵ greedy policy improvement occasionally tries something new/suboptimal. ϵ greedy doesn't work well for all tasks & even fails in some.
- Exploration (increase knowledge) do things not done before in hopes of getting higher reward. Take risks to learn about unknown options.
- Exploitation: do what you know to yield highest reward.
↳ maximise performance based on knowledge. take advantage of best options we know.
- Regret is used to derive an optimal strategy.
- Exploration problem made tractable using: Multi-Armed Bandits, MDPs

Multi-Armed Bandits (subset of RL) [simplified MDP as there are no states]

- defined as a tuple of $\langle A, R \rangle$
- N machines with reward prob / prob. distr. over rewards $\{\theta_1, \theta_2, \dots, \theta_N\}$

At each time step we take an action a_t on 1 slot machine & receive reward r_t .

- $A = \{\text{pull}_1, \text{pull}_2, \dots, \text{pull}_N\}$ is a set of actions against each slot machine.

True value of action: $Q(a) = E[r|a] = \theta$

- Rewards r are observed in a stochastic fashion. It is assumed rewards follow ~~as~~ some prob. distr. A reward at time t is sampled from this distr.

$r_t = R(a_t) = p(r|a) \rightarrow$ fixed but unknown
returns $\begin{cases} 1 & \text{with prob. } \theta_i = Q(a_t) \\ 0 & \text{" " } (1 - \theta_i) \end{cases}$

Goal: maximise cumulative reward $\sum_{t=0}^T r_t$

Regret

- action value $Q(a) = E[r|a]$ is mean reward for action a .
- optimal value Q^* of optimal action a^* is
 $Q^* = Q(a^*) = \max_{a \in A} Q(a) = \max_{i \leq N} \theta_i = \max_{a \in A} E[r_t | a_t = a]$
- Regret is used as a one step opportunity loss to reason about explo-exploit. trade
↳ $\text{regret} = \text{optimal value} - \text{actual value}$

loss function: total regret we have by not selecting the opt. action upto time step T .

$$L_T = E \left[\sum_{t=1}^T (Q^* - Q(a_t)) \right] = \sum_{a \in A} N_T(a) \Delta_a$$

Δ_a → per action regret/gap
 $N_T(a)$ → action selection counter

$N_T(a)$ → ^{expected} no. of times action a is selected.

using regret:

if we know optimal action with best reward then

Maximise cumulative rewards $\equiv$ min. total regret

- can ^{use} ~~analyse~~ regret to ~~explore~~ analyse diff. regret strategies.
- the sum for total regret extends beyond single episodes.
- good algo ensures small visitations for large action regrets.

strategies for bandits

No exploration

Explore at random

Smart exploration
- explore actions with high uncertainty

Random exploration: ϵ -greedy

$$a = \begin{cases} \underset{a \in A}{\operatorname{argmax}} Q_T(a) & \text{with prob. } 1 - \epsilon \\ \text{random action} & \text{with prob. } \frac{\epsilon}{\text{expected } |A|} \end{cases}$$

- ϵ greedy with const. ϵ has a linear total regret.
- we end up exploring bad actions again due to randomness.
- sol: 1) decrease ϵ over course of training [not easy to tune param]
- 2) be optimistic with options of high uncertainty: prefer actions which don't have a confidence value estimation yet. Idea also called Upper Confidence bounds (UCB).

UCB

idea: Estimate a UCB $U_t(a)$ for each action value, such that with a high probability we satisfy $Q(a) \leq \hat{Q}_t(a) + U_t(a)$.

then we select action that minimises UCB

$$a_t^{\text{UCB}} = \underset{a \in A}{\operatorname{argmax}} [\hat{Q}_t(a) + U_t(a)]$$

$\equiv$ selecting actions with low/no confidence value estimation

$\downarrow N_t(a) \rightarrow$ large bound $U_t(a)$ [estimated value is uncertain]

$\uparrow N_t(a) \rightarrow$ small " " [" " is certain/accurate]

- central limit theorem: uncertainty decreases as $\sqrt{N_t(a)}$

derivation through Hoeffding inequality gives us

$$U_t(a) = \sqrt{\frac{-\log p}{2 N_t(a)}}; \quad \left(p = e^{-2t U_t(a)^2} \right)$$

- with more samples we become more confident

use $p = \frac{1}{t}$

$$U_t(a) = \sqrt{\frac{\log t}{2N_t(a)}}$$

→ ensures we keep exploring

- select optimal action ~~as~~ more often as $t \rightarrow \infty$

Finally vanilla UCB uses

$$U_t(a) = \sqrt{\frac{2 \log t}{N_t(a)}}$$

$$A_t^{UCB} = \operatorname{argmax}_{a \in \mathcal{A}} Q(a) + \sqrt{\frac{2 \log t}{N_t(a)}}$$

- UCB doesn't assume any prior ~~distr.~~ on reward distr.
- Using prior knowledge on distr. allows for a better bound.

~~Probability~~ Matching. Bayesian UCB

- Bayesian bandits exploit prior knowledge of rewards $p[R]$
 - ↳ try compute posterior distr. $p[R|h_t]$
- $h_t = [a_1, r_1, \dots, a_{t-1}, r_{t-1}]$
- use posterior to guide exploration
- in Bayesian view start with prior over unknown params, here parameters that define unknown distr. over rewards for each arm.
- given observations/data, update uncertainty over the unknown param. using Bayes rule.

Ex: let reward of arm i depend on prob. distr. that depends on param ϕ_i

$$p(\phi_i | r_{1:t}) = \frac{\text{likelihood} \quad p(r_{1:t} | \phi_i) \quad \text{prior} \quad p(\phi_i)}{p(r_{1:t})} \quad \rightarrow \text{computing this req. additional structure or form of prior \& likelihood. (*)}$$

$$= \frac{p(r_{1:t} | \phi_i) p(\phi_i)}{\int_{\phi_i} p(r_{1:t} | \phi_i) p(\phi_i) d\phi_i}$$

(*) if parametric rep. of likelihood & prior are same then they are called conjugate. Ex: Beta distr. is conj for Bernoulli distr.

Ex: consider Bernoulli distr. with rewards $\in \{0, 1\}$

Prior: uniform $[0, 1]$ if $a \in \mathcal{A}$

posterior is a Beta distr. $\text{Beta}(\alpha_a, \beta_a)$ with initial params.

$\alpha_a = 1$ & $\beta_a = 1$ for each action a

Update the posterior

$$\begin{aligned} \alpha_a &\rightarrow \alpha_a + 1 & \text{if } r_t = 1 \\ \beta_a &\rightarrow \beta_a + 1 & \text{if } r_t = 0 \end{aligned}$$

Exploration via Prob. Matching

via Thompson sampling

1) $Q(a)$ follows a Beta distr. for Bernoulli Bandit

2) init prior (ex: $\alpha = \beta = 1$)

3) At each time step t , sample an expected reward $\tilde{Q}(a)$ from prior Beta (α_i, β_i) for every action.

↳ select & execute the best action amongst the samples

$$a_t^{TS} = \underset{a \in \mathcal{A}}{\operatorname{argmax}} \tilde{Q}(a)$$

4) with new experience update Beta distr.

$$\alpha_i \rightarrow \alpha_i + r_i \mathbb{I}[a_t^{TS} = a_i]$$

$$\beta_i \rightarrow \beta_i + (1 - r_i) \mathbb{I}[a_t^{TS} = a_i]$$

Model Based RL

Why?

- inspired by how organisms create internal models to predict & plan.

Adv. over Model Free RL

- Better data efficiency
- faster adaptation to changes
- improved exploration & safety

What is a model? i/p?

A model is a representation that explicitly encodes knowledge about the structure of the env & task.

Encodes

Transitions/dynamics model

$$s_{t+1} = f_s(s_t, a_t)$$

reward model

$$r_{t+1} = f_r(s_t, a_t)$$

} MBRL

$$a_t = f_s^{-1}(s_t, s_{t+1}) \rightarrow \text{inv-dyn. model}$$

$$d_{ij} = f_d(s_i, s_j) \rightarrow \text{dist model}$$

$$G_t = Q(s_t, a_t) \text{ or } G_t = V(s_t) \rightarrow \text{future rewards model}$$

i/p: states, observations, latent states $\rightarrow$ extract features from raw observations & use them as i/p.

How Do we use a Model?

Background planning

I) Env. Data augmentation/simulation

① Dyna-Q (Dyna Architecture)

- use collected data to learn transition & reward model
- train traditional RL algo (ex: Q-learning) using both ~~real~~ env. data (real experience) & data generated from learned model (simulated exp.)

② Model Based Policy Optimisation (MBPO)

- use collected data to learn $p_\theta(s', r | s, a)$ i.e. transition model
- Apply traditional PO methods on synthetic model rollouts.
- take action in real env.

③ Domain Randomization

- used for sim-to-real transfer
 - Train an RL Model on available simulator $\rightarrow$ learn a good policy on a 'distr of similar env' differing in some physical parameters
- In this way real system is just another variation for policy.

II) Sample efficient Policy learning

idea: train model & policy jointly end-to-end

Goal: maximise reward of parametric policy $J(\theta) = \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)$

with $a_t = \pi_{\theta}(s_t)$ & $s_{t+1} = T(s_t, a_t)$

- just use policy gradient methods. Calculate $\nabla_{\theta} J(\theta)$ by ex: REINFORCE
 - minimises real-world interaction needed to train on effective policy.
 - When models are smooth (differentiable), it's possible to calc derivatives that show how small changes in actions affect next state & how small changes in states affect rewards.
- This allows end-to-end differentiation through back-prop, crucial for sample efficiency. Backprop. through model to optimise policy.

Algo: (1) Run base policy $\pi_0(a_t | s_t)$ (Ex: random policy) to collect data samples $\mathcal{D} = \{(s, a, s')\}$

2) Learn dynamics model $f_{\theta}(s, a)$ by minimising $\sum_i \|f_{\theta}(s_i, a_i) - s'_i\|^2$

3) Backprop. through $f_{\theta}(s, a)$ into policy to optimise $\pi_{\theta}(a_t | s_t)$

4) Run $\pi_{\theta}(a_t | s_t)$

5) Append visited tuples (s, a, s') to $\mathcal{D}$

+ long term credit assignment
: attribute reward/penalties to actions over extended periods

- vanishing/exploding gradients, similar to training RNNs

- prone to local minima

+ data efficiency: differentiable transition & reward models

- fixed dynamics: env. dynamics are inherent & can't be altered.

+ deterministic & low var

+ principles behind BPTT well understood.

Background Planning

- learn to act in any situation

- optim. variables: shared param. θ

$$s \sim J(\theta) = E_{s_0} \left[\sum_{t=0}^H \gamma^t r_t \right]$$

$$a_t = \pi_{\theta}(s_t)$$

Decision-Time Planning

- Find best seq. of actions for current situation/state.

- optim. variables: $a_0, a_1, \dots, a_H$
seq. of actions and/or states

$$J(a_0 \dots a_H) = \sum_{t=0}^H \gamma^t r_t$$

- We look at prob. with discrete actions and states. Their dimensionality is large & is impossible to enumerate the solutions.

Monte Carlo Tree Search (MCTS)

- Used for online planning: we need a transition model or we need a way to simulate.

steps: Select $\rightarrow$ Expand $\rightarrow$ Simulate $\rightarrow$ Backup

1) Selection

- Follow a policy π_s to do selection in an MDP
- In Nodes seen before select action according to UCT (UCB/Bandit)

$$a \sim V_i + C \sqrt{\frac{\ln(N)}{n_i}}$$

$V_i \rightarrow$ value estimate at node i

$C \rightarrow$ tunable exploration param.

$N \rightarrow$ No. of visits of parent node

$n_i \rightarrow$ " " " at node i

- do it until reaching leaf node.

2) Expansion

- we know transition model so we know where actions let's us end up in
- else do some one-step simulations to know successor states.

3) Simulation

- Randomly choose a new child node
- play randomly (with random rollout policy π_r) until game finishes.
- Doing all the MC rollouts gives $\hat{Q}(s,a)$

4) Backup

- we now have estimates of $\hat{Q}(s,a)$ of leaf nodes
- Backup Q values & no. of visits all way up to the root of the tree

- we used 2 policies

1) selection policy π_s (uses Q -values where we have knowledge about)

2) random sim " π_r (for leaves onwards where we don't know anything)

- update π_s now. It gets deeper through iterations & more accurate with backing up info of leaf nodes over time.

- can run MCTS forever

- To integrate planning & execution (1) Run MCTS until convergence starting from current real world state.

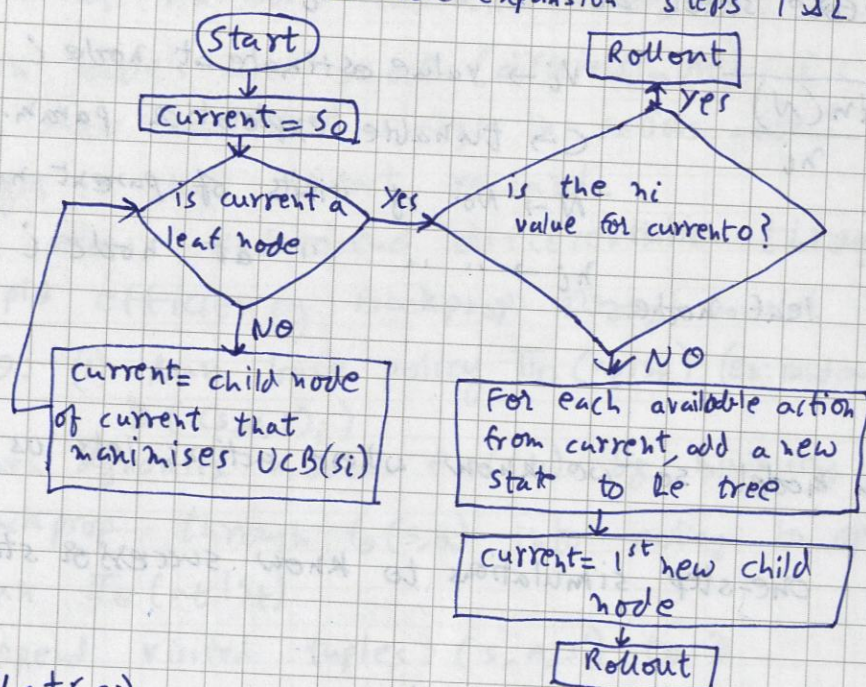
- 2) Take best action in real world
 3) go back to 1

UCT A value range of 0

$$UCT: a = \operatorname{argmax}_i V_i + c \sqrt{\frac{\ln(N)}{n_i}}$$

in many cases: $a = \operatorname{argmax}_i \frac{w_i}{n_i} + c \sqrt{\frac{\ln(w_i)}{n_i}}$

Algo for tree traversal & node expansion steps 1 & 2



Rollout(s_i):

Loop forever:

if s_i is a terminal state
 return value(s_i)

$A_i =$ random (available actions(s_i))

$s_i =$ simulate (A_i, s_i)

